Q1. DBMS vs Traditional File System

A **traditional file system** stores data as raw files managed by the OS. Each application that uses the data typically defines its own format, leading to the same data being stored in multiple files, copies getting out of sync, and no built-in support for concurrent access or recovery.

A **DBMS** provides a centralised, structured way to store, query, and manage data with built-in support for concurrency control, data integrity enforcement, access control, and recovery from failures.

Example: In a file system, departments like registration, billing, and pharmacy keep separate, disconnected records. If a patient moves, their address might update in one place but not the others, leading to errors. A DBMS solves this by providing a single source of truth. When data is updated, it reflects everywhere immediately, ensuring consistency and safe concurrent access.

Q2.

A **strong entity** has a primary key made entirely of its own attributes ,it can be uniquely identified without referencing any other entity.

A **weak entity** cannot be uniquely identified by its own attributes alone. It depends on an **identifying (owner) entity** for its existence and identity.

How the primary key is formed: The weak entity's primary key = its **partial key** (discriminator) + the **primary key of the owner entity**.

The relationship is called an **identifying relationship** (shown with a double diamond in ER diagrams).

Q3.

Model it as a **multivalued attribute**.

A simple attribute holds exactly one value. Since a student *may have multiple* phone numbers, a single attribute can't represent this.

A **weak entity** lacks the attributes necessary for unique identification on its own and instead depends on an **identifying (owner) entity** for both its identity and existence.

The primary key of a weak entity is constructed by combining its own **partial key** with the **primary key of the owner entity**.

This connection is known as an **identifying relationship** and is represented in ER diagrams by a double diamond symbol.

Q4.

A **super key** consists of any set of attributes that can uniquely identify a tuple, even if it includes extra, redundant data.

A multivalued attribute represents a set of values for a single entity instance. This directly models the "zero or more phone numbers per student" reality.

Q4.

- A **super key** is any set of attributes that uniquely identifies a tuple. It may contain redundant attributes.
- A **candidate key** is a *minimal* super key; no attribute can be removed without losing the uniqueness property.

Relation: `Student(roll_no, email, name, branch)`

Assumptions: roll_no is unique per student, email is unique per student, name and branch are not unique.

Candidate keys:

- `{roll_no}` ,uniquely identifies; removing anything leaves nothing → minimal
- `{email}` ,same reasoning

Super keys that are NOT candidate keys

- `{roll_no, email}` ,unique but redundant; drop either and still unique
- `{roll_no, name}` ,unique (roll_no alone suffices); non-minimal
- `{roll_no, branch}` ,same
- `{email, name}` ,non-minimal

Key relationship: Every candidate key is a super key, but not every super key is a candidate key.

Q5. Foreign Key Constraint Violation

Answer:

- **Constraint violated:** Foreign Key constraint
- **What it protects against:** It prevents "orphan" rows, records that reference a parent entity that doesn't exist. In this case, an enrollment record pointing to a non-existent student would make the database inconsistent (you couldn't look up who enrolled).
- **What should happen:** The **insert should be rejected** (the DBMS raises an error and rolls back the operation). The row is not inserted.

Q6. Mapping M:N Relationships

- **Additional structure required:** A **junction table** (also called associative table, bridge table, or relationship table) must be created.
- **Its primary key consists of:** The combination of the **foreign keys referencing both participating entities**,i.e., a composite primary key made of the PKs of both tables.
- **Why not just a column in one table?** If you put it in one table, you can only store one value per row. For M: N, entity A can relate to *many* B's, and entity B can relate to *many* A's. A single column can't hold multiple values cleanly; you'd need repeating groups, which violates 1NF (First Normal Form). The junction table gives each pairing its own row.

Part II: Long Answer

Part A, Entity and Relationship Identification

Entities and key attributes:

Entity	
Customer	customer_id (PK), name, email, phone
Restaurant	restaurant_id (PK), name, address, cuisine_type
MenuItem	item_id (PK), name, price, description
Order	order_id (PK), order_date, total_amount, status

Relationships:

Relationship	Cardinality	Justification
Customer **places** an order	1:N	One customer can place many orders, but each order belongs to exactly one customer
Order **placed at** the restaurant	N:1 (or 1:N from the restaurant's side)	Each order is associated with exactly one restaurant; a restaurant can have many orders
Order **contains** MenuItem	M: N	One order can include multiple menu items; one menu item can appear in multiple orders
Restaurant **offers** MenuItem	1:N	A restaurant can offer many menu items; each menu item belongs to one restaurant

Part B , Attribute Classification

Multivalued attribute: `Customer.phone`,a customer may have multiple phone numbers (mobile, home, work). It holds a *set* of values, not just one.

Composite attribute: `Restaurant.address`,an address is naturally decomposable into sub-parts: street, city, state, and pin_code. Each sub-part has independent meaning and may be queried separately (e.g., "all restaurants in Kanpur").

Part C , Junction Table for Order ↔ MenuItem

- **Primary key:** `(order_id, item_id)` ,composite PK; the combination uniquely identifies each line item in an order
- **Foreign keys:**
  - `order_id` → references `Order(order_id)`
  - `item_id` → references `MenuItem(item_id)`
- **Additional attribute:** `quantity` ,required because "each order may contain multiple menu items *each with a specified quantity*" (given in the scenario). This is a **relationship attribute** ,it doesn't belong in Order or MenuItem alone, only in the pairing.

Part III: ER Diagram + Justification

Question 1: Hospital ER Diagram

You need to draw this in draw.io. Here's the exact specification to build it:

Entities and attributes:

Patient (rectangle)

- patient_id (PK, underlined)
- name
- date_of_birth
- gender
- diagnosis

Doctor (rectangle)

- doctor_id (PK)
- name
- specialization
- phone

Ward (rectangle)

- ward_no (PK)

- ward_name
- capacity

Relationships:

1. **Admitted In** between Patient and Ward → **1:N** (many patients, one ward)
    - Patient side: **total participation** (double line) , every patient must be admitted to a ward
    - Ward side: **partial participation** (single line) , a ward can exist even if currently empty
2. **TreatedBy** between Patient and Doctor → **M:N**
    - Both sides: partial participation (a doctor can exist without current patients; a patient could theoretically be in the system before treatment begins, though you could argue total on the patient side)

ER notation reminders for draw.io:

- Rectangles = entities
- Ellipses = attributes
- Underlined text in ellipse = primary key attribute
- Diamond = relationship
- Double line between entity and relationship = total participation
- Single line = partial participation
- "1" and "N" or "M" labels on relationship lines

Question 2: Design

The Doctor–Patient relationship is M:N because a single patient may be treated by multiple doctors during their hospital stay (e.g., a surgeon and an anaesthetist), and a single doctor simultaneously treats multiple patients. This M:N relationship creates an associative entity **Treatment** (or Treats), whose composite primary key consists of `(doctor_id, patient_id)`. Additional attributes such as `treatment_date` or `notes` may be added to this entity.

Regarding participation: the Patient–Ward relationship has **total participation on the Patient side** ,every patient in the system must be admitted to a ward; a patient record without a ward assignment would be operationally meaningless. The Ward side has **partial participation** ,a ward can exist in the database (for administrative purposes) even if no patients are currently assigned to it.

For attribute design, I modelled `address` in the Restaurant entity as a **composite attribute** (street, city, state, pincode) rather than a single string. This allows targeted querying, for example, filtering restaurants by city ,which a flat string attribute would not support efficiently.